Project Proposal: WinTask

HP-ULL Priority-Based Access Task Schedular for Windows

Course: Operating Systems (Lab Project)
Semester: 5th
Program/Section: SE-5A
Name: Muhammad Bin Moazzam
CMS ID: 3569-2024

Overview:

WinTask is a Windows-native C++ proof of concept that models a server-oriented, high-volume request-processing system. It accepts requests through a dedicated ingestion thread and routes them by severity into three independently bounded priority queues: High, Medium and Low. A dedicated consumer processes each queue, while atomic synchronization, fixed-capacity storage and load shedding are used to keep request handling bounded under pressure. The system also demonstrates application-level file and folder authorization and records request outcomes and ingress-to-completion latency for offline analysis. The project has a stretch objective of evaluating whether the design can complete up to **1,000,000 requests per second** under a defined synthetic workload; this is a benchmark target, not a guaranteed performance claim.

Responsibilities:

- **Prioritize and Route:** Classify requests by severity: 5 to High, 3–4 to Medium, and 1–2 to Low; preserve FIFO order within each priority queue.
- **Manage Bounded Resources:** Store each request in a compact 32-byte slot and use preallocated ring buffers with a combined capacity of 1,310,720 slots.
- **Coordinate Concurrent Processing:** Use one ingestor and three dedicated consumer threads with single-producer/single-consumer queue ownership and explicit atomic memory ordering.
- **Control File Access:** Apply predefined per-request application-level policies to permitted file and folder operations within a controlled test workspace.
- **Maintain Stability:** Reject requests when their destination queue is full, record capacity rejections, and avoid overwriting pending or in-flight requests.
- **Measure and Report:** Collect request status and ingress-to-completion latency in preallocated diagnostics arrays, then export results to CSV after processing has stopped.

Core Architecture Design

WinTask combines compact request records, priority-separated SPSC ring buffers, dedicated consumers, application-level authorization and out-of-band diagnostics to model a bounded server-side request-processing pipeline.

1. Request Representation: 32-Byte Queue Slot

- **Purpose:** Keep request metadata and its fixed-size payload together in one compact, predictable memory record.
- **Mechanism:** A 16-byte `JobDescriptor` and a 16-byte `JobPayload` form one 32-byte `QueueSlot`. The descriptor contains the job type, severity, urgency, packed state/flags, job ID and 64-bit ingress QPC timestamp. The payload contains the user ID, resource-policy ID and 64-bit synthetic session-signature value.
- **Memory Layout:** The queue allocation begins on a 64-byte boundary, allowing two 32-byte slots to occupy one assumed 64-byte cache line. Compile-time size and offset assertions verify the layout; packing directives are not required.

2. Priority Layer: Three Bounded SPSC Ring Buffers

- **Purpose:** Separate High, Medium and Low requests so each priority tier has independent capacity and a dedicated consumer.
- **Mechanism:** High uses 262,144 slots; Medium and Low each use 524,288 slots. Capacities are powers of two, so slot indexing uses `counter & (Capacity - 1)`.
- **Capacity:** The queues reserve 1,310,720 slots in total, equivalent to 40 MiB of 32-byte request storage. The additional 310,720 slots are distributed headroom, not a shared overflow reserve.

3. Ingestion and Load-Shedding Layer

- **Purpose:** Route incoming work quickly while preventing a full queue from corrupting or overwriting pending requests.
- **Mechanism:** One ingestor validates basic request fields, captures the QPC timestamp, selects the destination queue from severity and publishes the complete slot. If that queue has no free capacity, the request is rejected and a simulated HTTP 429 capacity-rejection event is counted; no actual HTTP endpoint is implemented.
- **SIMD Evaluation:** AVX2 may be used to copy one 32-byte slot with one 256-bit store. Two slots require two 256-bit stores. SIMD is evaluated as an optimization rather than assumed to complete in one clock cycle.

4. Concurrency and Worker-Execution Layer

- **Purpose:** Process all three priority queues concurrently while keeping queue ownership simple and bounded.
- **Mechanism:** The ingestor is the sole producer for all queues. Three dedicated consumers independently process High, Medium and Low. The producer owns each tail counter, while the corresponding consumer owns its head counter. Release/acquire atomic operations publish complete slots and release them for reuse.
- **Processor Affinity:** The target-machine configuration assigns the ingestor to Core 0 and the High, Medium and Low consumers to Cores 2, 4 and 6 respectively. The validator and benchmark apply the same configured pinning for relevant test threads.

5. Job Processing and File-Access Authorization

- **Purpose:** Demonstrate distinct request workloads and ensure file-facing operations follow an explicit access policy.
- **Job Types:** `VALIDATE_SESSION` checks a synthetic session record; `TELEMETRY_LOG` creates a telemetry/event record; and `COMPUTE_SIGN` performs deterministic 64-bit rotation/mixing work. The bit-scrambling workload is synthetic and is not cryptographic authentication.
- **Authorization:** The request user ID and resource-policy ID identify a predefined application-level policy. The authorization component checks the permitted operation and approved test workspace before a file-facing handler proceeds. This does not provide OS-level isolation against unrelated processes.
- **Urgency:** The urgency field is retained as request metadata and recorded in diagnostics. The initial POC preserves FIFO order within each queue and does not introduce intra-queue urgency reordering.

6. Diagnostics and Reporting Layer

- **Purpose:** Measure completed work and latency without adding file formatting or analytical reads to the queue hot path.
- **Mechanism:** Each consumer writes results to its own preallocated diagnostics array. After all accepted work drains and workers terminate, WinTask exports `TestResults.csv` with job ID, priority class, urgency, latency in microseconds and terminal status.
- **Accounting:** The benchmark distinguishes offered, accepted, successfully completed, capacity-rejected and failed requests. Completed throughput is calculated from successfully completed requests divided by the measured interval.

Operations / Queries

Core processing contract: WinTask request pipeline

Operation / Module	Component Used	Purpose
Ingest and route	Ingestor + severity mapping	Validates a request, captures its ingress timestamp and selects the High, Medium or Low queue.
Publish request	SPSC ring buffer	Writes a complete 32-byte slot and publishes the tail using release semantics.
Consume request	Dedicated consumer	Observes published work, processes its assigned priority queue and releases completed slots.
Authorize operation	Authorization policy table	Checks user/resource-policy identity and the requested file operation against the controlled workspace policy.
Record result	Consumer-isolated diagnostics array	Stores terminal status and ingress-to-completion latency without writing CSV on the hot path.
Export results	Post-run reporting	Writes per-request results and aggregate offered/accepted/completed/rejected/failed counts to the output file.

Design Rationale

The architecture is intentionally bounded and specialized for a single-producer, three-consumer workload. It avoids the additional coordination costs of a generic multi-producer/multi-consumer queue, dynamic worker creation and a global overflow queue. Its fixed memory footprint, independent priority queues and explicit rejection policy make overload behavior measurable and testable.

Layer	Structure / Mechanism	Purpose in WinTask
Request storage	32-byte `QueueSlot`	Keeps metadata and payload compact, fixed-size and cache-conscious.
Priority routing	Three SPSC ring buffers	Separates priority tiers and gives each queue one dedicated consumer.
Synchronization	Atomic head/tail counters	Uses release/acquire ordering to publish requests and reclaim slots safely.
Resource protection	Bounded capacity + load shedding	Prevents unbounded memory growth and records requests rejected at queue capacity.
File authorization	User/resource-policy lookup	Restricts WinTask-mediated file operations to configured permissions and test paths.
Measurement	QPC + preallocated diagnostics	Provides ingress-to-completion latency and completed-throughput reporting.

Implementation Plan

1. **Data Model and Configuration:** Implement `WinTask.Types.hpp` and `WinTask.Config.hpp` with job enums, the 32-byte descriptor/payload/slot layout, compile-time assertions, queue capacities, masks, priority mapping and configured processor assignments.
2. **SPSC Queue Implementation:** Implement `WinTask.SPSCQueue.hpp` with preallocated storage, 64-byte-aligned queue allocation, independently aligned head/tail controls, power-of-two indexing and release/acquire publication and reclamation.
3. **Platform, Authorization and Diagnostics:** Implement `WinTask.Platform.hpp` for QPC and thread affinity, `WinTask.Authorization.hpp` for controlled file/folder policy checks, and `WinTask.Diagnostics.hpp` for preallocated result arrays, counters and post-run CSV export.
4. **Integrated Request Engine:** Implement `WinTask.Engine.hpp` to route requests, enforce queue-full rejection, run the three dedicated consumers, invoke the predefined job handlers and drain accepted work during shutdown.
5. **Validation and Benchmark Demonstration:** Build `WinTaskValidator.cpp` for structural, routing, queue, synchronization and authorization tests. Build `WinTaskBenchmark.cpp` for pinned end-to-end workload generation, completed-throughput measurement, latency collection and `TestResults.csv` generation.

Demonstration

The demonstration will submit a controlled synthetic workload containing all three priority classes and job types. It will show correct severity routing, concurrent processing by the dedicated consumers, application-level allow/deny decisions, capacity rejection when a queue is full, and final reconciliation of offered, accepted, completed, rejected and failed requests. The benchmark will report whether the one-million-completed-requests-per-second objective is reached under the stated machine configuration and workload; no performance result is assumed in advance.